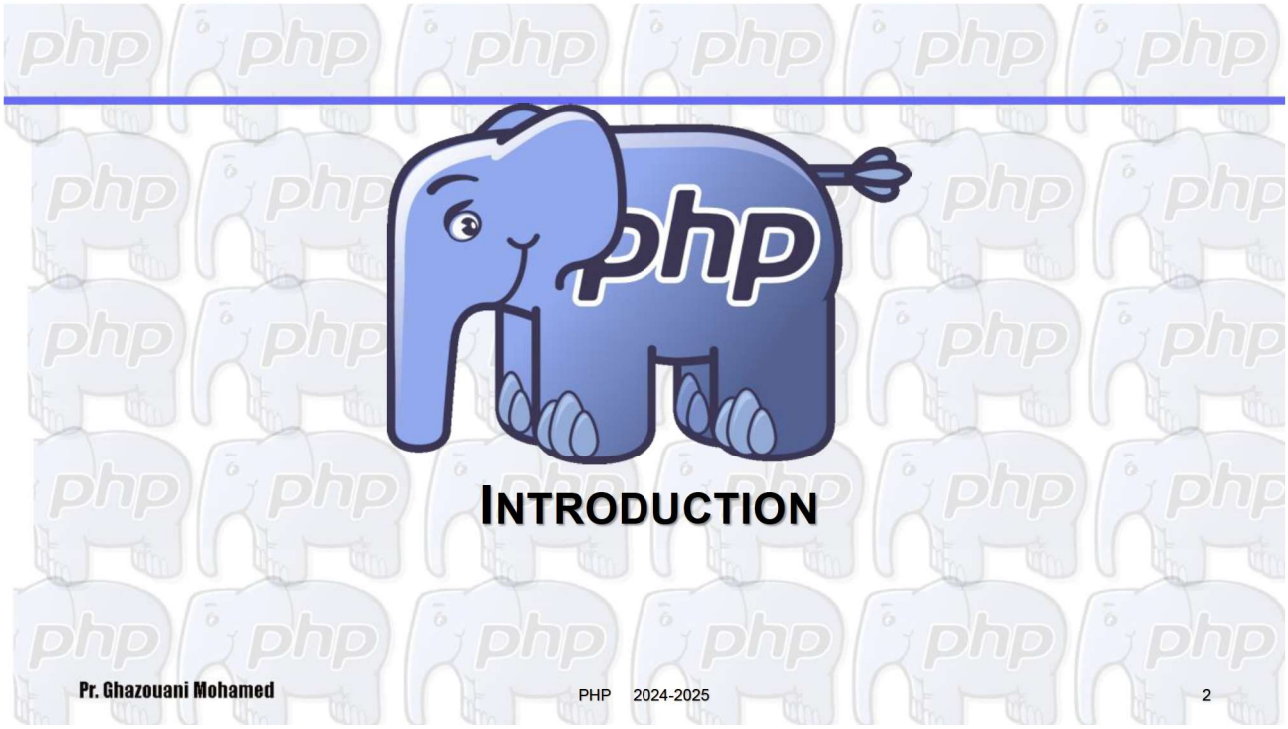



# Langage de Script

**Pr. Ghazouani Mohamed**

2024-2025

1




## INTRODUCTION



2

## Qu'est-ce que PHP ?

- Langage de script. Principalement utilisé côté serveur
- Créé en 1994-1995 par Rasmus Lerdorf
- Acronyme initial : Personal Home Page
- Acronyme récursif : PHP: Hypertext Preprocessor
- Langage multi plate-forme (UNIX / Windows…)
- Open Source
- ≈80% des serveurs Web (https://w3techs.com, 03/2022)
- Versions actuelles : 8.0.x, 8.1.x, 8.2.x, 8.3.x



3

## Langage de script

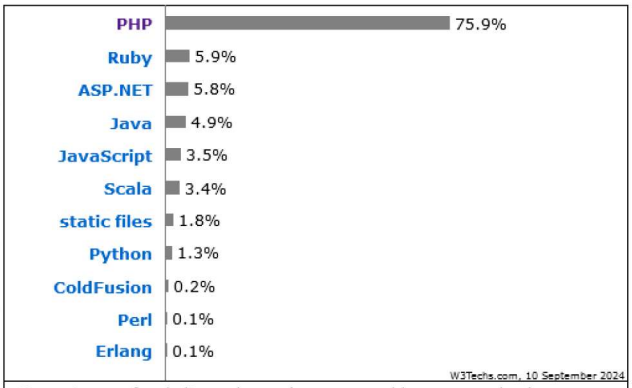
- Langage impératif, il suit un style de programmation "procédural"
- Langage interprété (exécuté ligne par ligne par un logiciel appelé interpréteur PHP)
- Pas de compilation
- Typage faible
- Programmation orientée objet possible



4

# Utilité et utilisation de PHP

- Création de contenus « dynamiques » Pages Web ou tout contenu
- Interface entre un serveur Web et des bases de données
- Création d'applications Web
- Le plus utilisé sur serveur



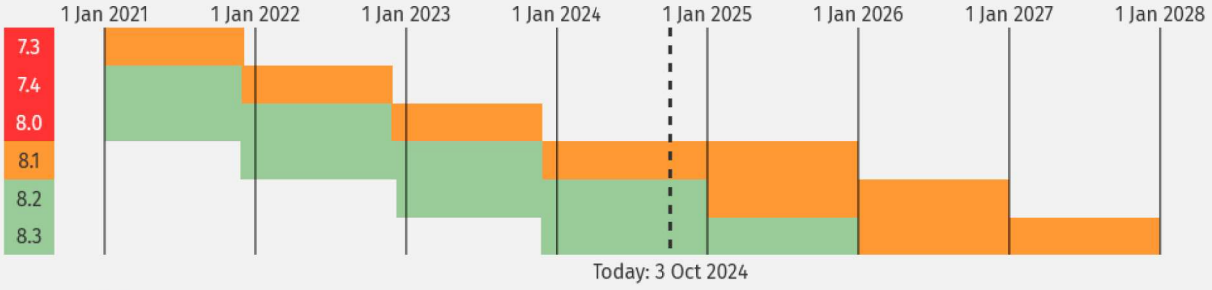
Percentages of websites using various server-side programming languages
Note: a website may use more than one server-side programming language

Source : https://w3techs.com/technologies/overview/programming_language



5

# Cycle de vie des versions de PHP




Key

2 ans

1 an

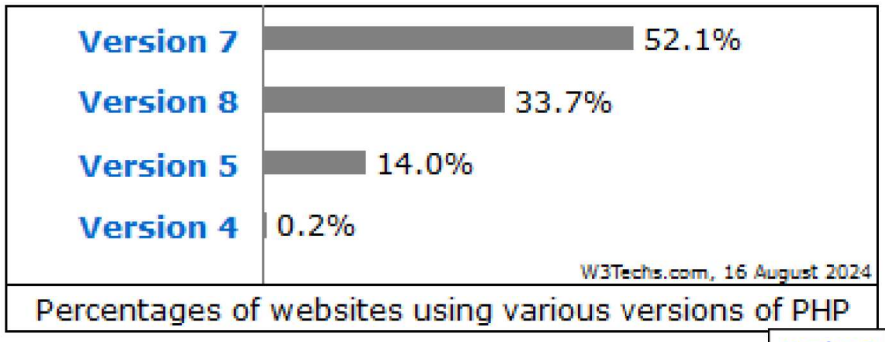
| | |
|---|---|
| Active support | A release that is being actively supported. Reported bugs and security issues are fixed and regular point releases are made. |
| Security fixes only | A release that is supported for critical security issues only. Releases are only made on an as-needed basis. |
| End of life | A release that is no longer supported. Users of this release should upgrade as soon as possible, as they may be exposed to unpatched security vulnerabilities. |

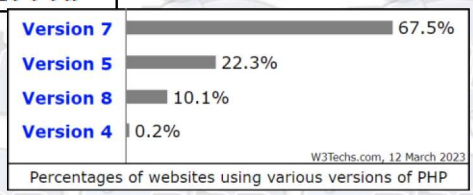
Source : https://www.php.net/supported-versions.php



6

## Versions de PHP en production dans le monde




Source : https://w3techs.com/technologies/details/pl-php



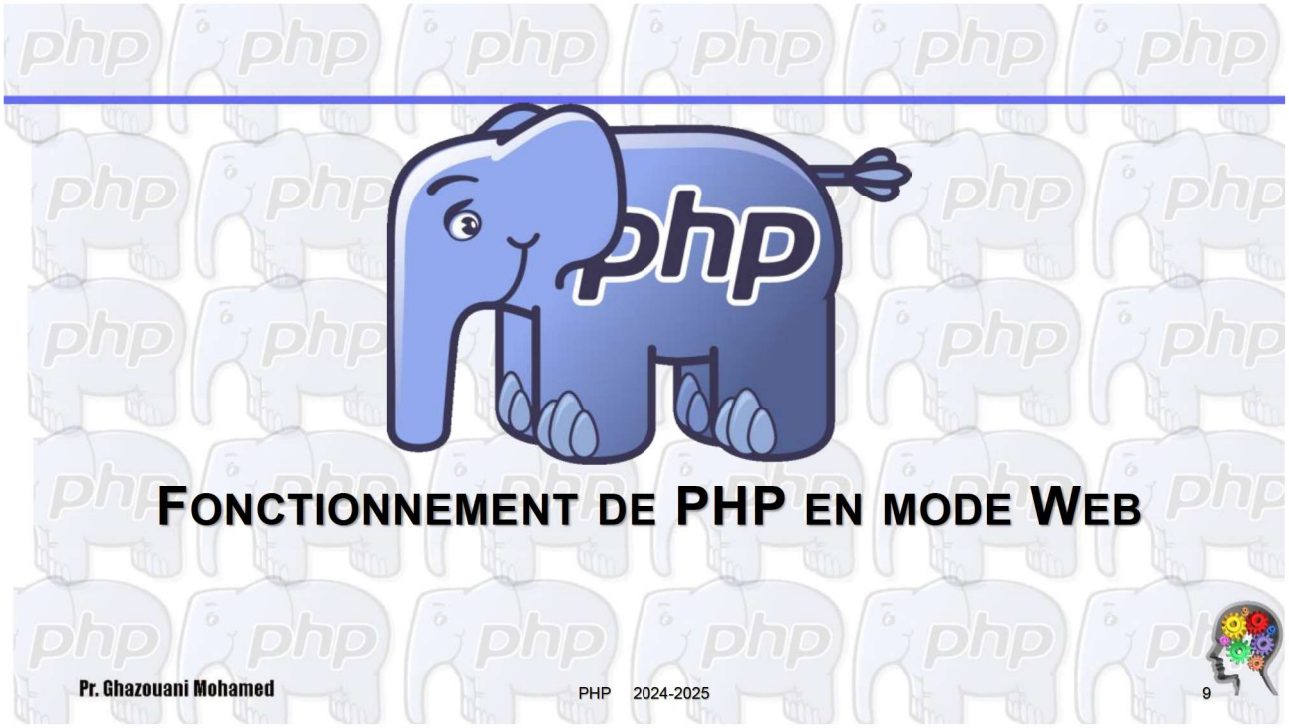
7

## Principales fonctionnalités de PHP

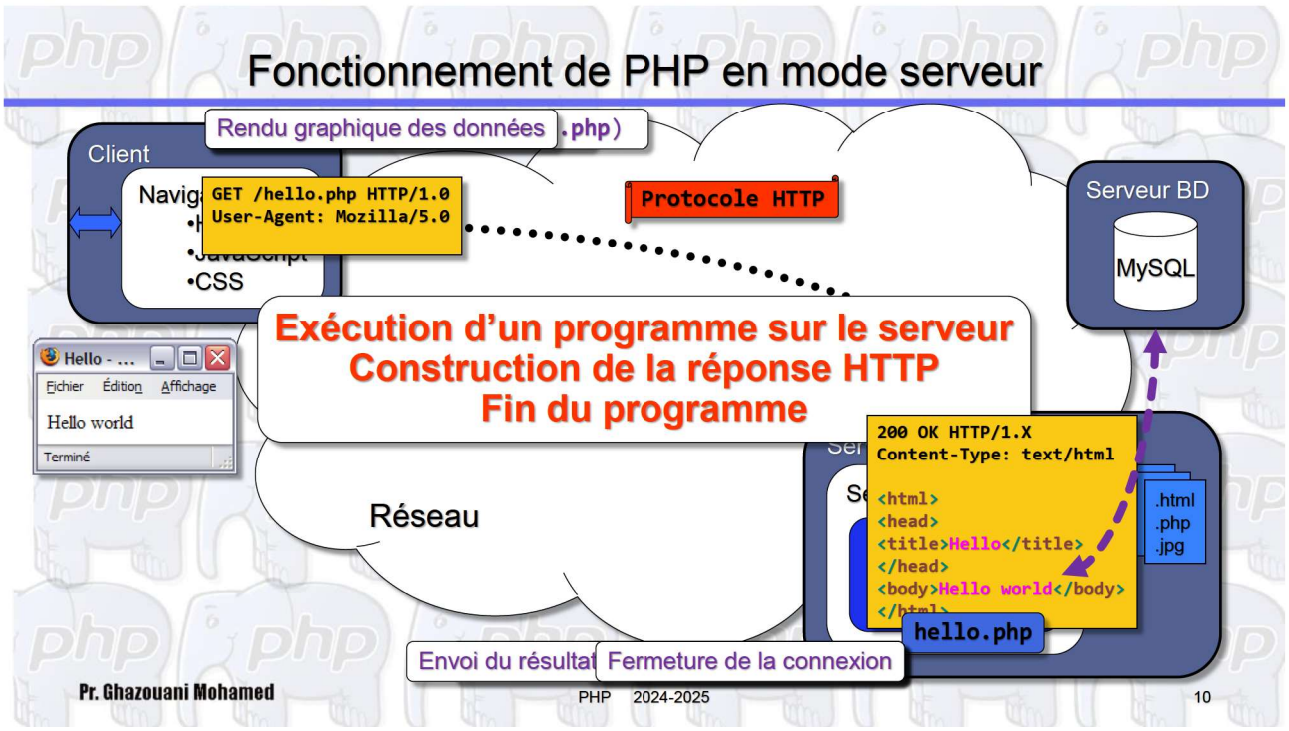
- Manipulation de chaînes et tableaux
- Calendrier / dates / heures
- Fonctions mathématiques
- Expressions régulières
- Bases de données (MySQL, PostgreSQL, Oracle, …)
- Accès au système de fichiers
- Manipulation d'images (Redimensionnement d'images et Conversion )
- HTTP / FTP / IMAP
- XML
- …



8

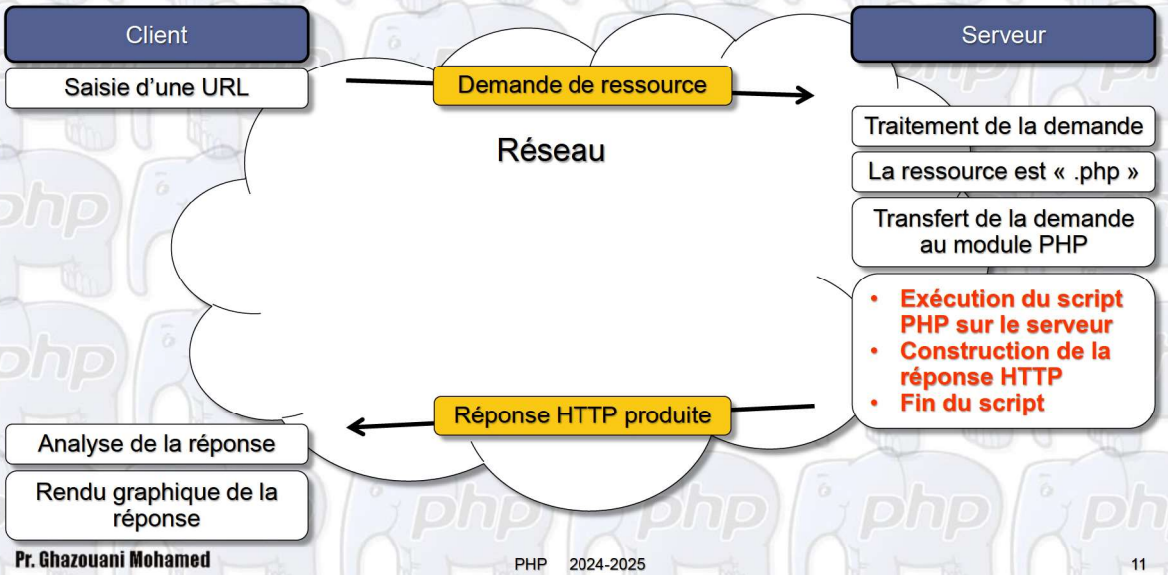

FONCTIONNEMENT DE PHP EN MODE WEB
Pr. Ghazouani Mohamed
PHP 2024-2025
9

9


Fonctionnement de PHP en mode serveur
Rendu graphique des données .php)
Client
Navig
•CSS
GET /hello.php HTTP/1.0
User-Agent: Mozilla/5.0
Protocole HTTP
Serveur BD
MySQL
Hello - ...
Fichier Édition Affichage
Hello world
Terminé
Exécution d'un programme sur le serveur
Construction de la réponse HTTP
Fin du programme
200 OK HTTP/1.X
Content-Type: text/html
<html>
<head>
<title>Hello</title>
</head>
<body>Hello world</body>
</html>
.html
.php
.jpg
hello.php
Réseau
Envoi du résultat
Fermeture de la connexion
Pr. Ghazouani Mohamed
PHP 2024-2025
10

10

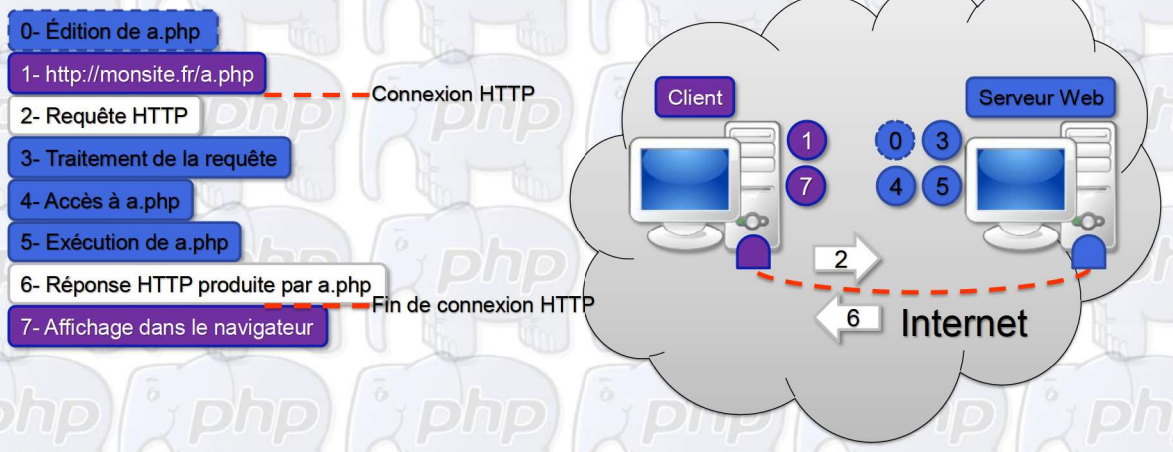
## Fonctionnement de PHP en mode serveur






11

## Fonctionnement de PHP en mode serveur Web





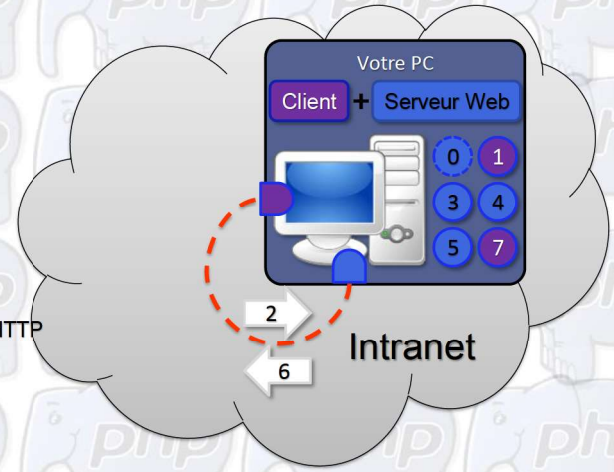

12

## Fonctionnement de PHP en mode serveur local

0- Édition de a.php
1- http://localhost:8080/a.php
2- Requête HTTP
3- Traitement de la requête
4- Accès à a.php
5- Exécution de a.php
6- Réponse HTTP produite par a.php
7- Affichage dans le navigateur

Connexion HTTP

Fin de connexion HTTP






13

## Programme en PHP

Délimitation du code PHP dans le fichier **.php** :

- `<?php` Code PHP

Fermeture optionnelle et déconseillée

- ~~`<?` Code PHP `?>`~~

Dépend de la configuration du serveur
➔ à bannir !! `short_open_tag`

- ~~`<script language="PHP">`~~
  Code PHP
  ~~`</script>`~~
- ~~`<%` Code PHP `%>`~~

Supprimés en PHP 7…



14

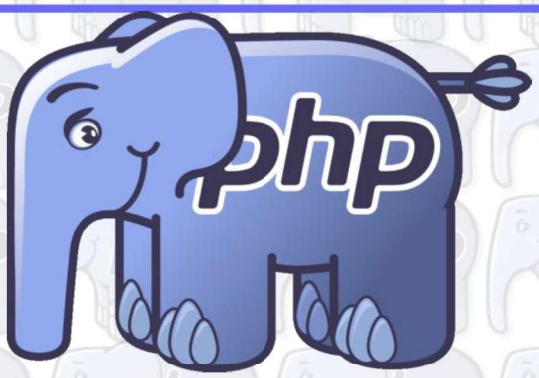
## Eléments de syntaxe PHP

- La syntaxe de PHP est celle de la famille « C » (C, C++, Java, …)
- Chaque instruction se termine par « ; »
- Commentaires:

```
/* jusqu'au prochain */
// jusqu'à la fin de la ligne
# jusqu'à la fin de la ligne
```



15




## VARIABLES ET TYPES



16

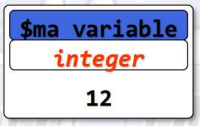
## Les variables et les types de données

- Tout identificateur commence par « $ »
- Les types sont :
  - Numérique entier : 12 ou réel : 1.54
  - Chaîne: "Hello" ou 'Bonjour'
  - Booléen: true, false
  - Tableau: $tab[2]=12
  - Objet : $utilisateur = new Personne();
  - Ressource : $fichier = fopen("mon_fichier.txt", "r");
  - NULL : $age = null;
- Les variables ne sont pas explicitement déclarées



## Les variables et les types de données

- La « déclaration » d'une variable correspond à sa première affectation
- Le type d'une variable est dynamique et est déterminé par la valeur qui lui est affectée
- Une variable possède donc :
  - Un nom (commençant par $)
  - Un type dynamique
  - Une valeur



## Typage à l'affectation. Exemple

```php
// Pas de déclaration préalable de variable

$test = 1.5;

$test = 12;

$test = array();

$test = '10';
```



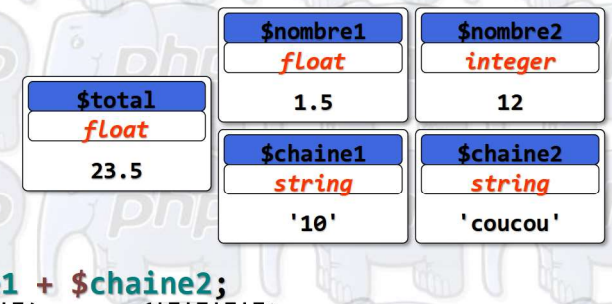


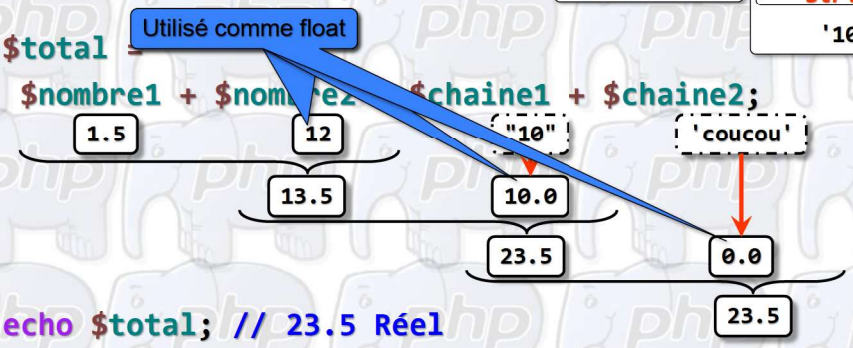

19

## Typage en fonction du contexte. Exemple

```php
$nombre1 = 1.5;
$nombre2 = 12;
$chaine1 = "10";
$chaine2 = 'coucou';
$total =
 $nombre1 + $nombre2 $chaine1 + $chaine2;

echo $total; // 23.5 Réel
```






20

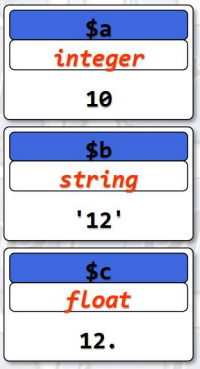
## Modification de types

- Le type désiré peut être précisé : cast
- `$variable = (nom_du_type) valeur`

```php
$a = (integer) '10';

$b = (string) 12;

$c = (float) $b;
```






21

## Définition de constantes

```php
<?php
define('MA_CONTANTE1', 'Bonjour');
```

nom

valeur

Définition d'une constante

Par convention, les constantes sont nommées en SCREAMING_SNAKE_CASE

```php
const PI = '3.14';
```

nom

valeur

Utilisation de la constante

```php
echo MA_CONTANTE1;
```



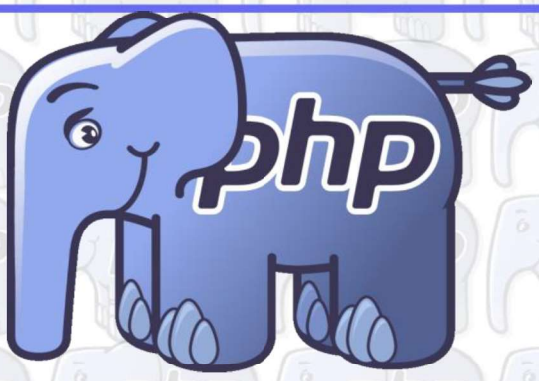
22

## Conventions de nommage pour les variables

En programmation, il existe plusieurs conventions de nommage pour les variables, en fonction du langage utilisé et des préférences de l'équipe de développement. Voici quelques-unes des plus courantes :

- **camelCase** : La première lettre est en minuscule et chaque mot suivant commence par une majuscule.
  **Exemple : maVariable, prixUnitaire.**
- **PascalCase** : Chaque mot commence par une majuscule, y compris le premier.
  **Exemple : MaVariable, PrixUnitaire.**
- **snake_case** : Tous les mots sont en minuscules et séparés par des underscores.
  **Exemple : ma_variable, prix_unitaire.**






## CHAÎNES DE CARACTÈRES

## Substitution/interprétation de variables dans les chaînes

- Guillemets doubles

```
$a="chaîne";
$b="voici une $a";
```

```
chaîne
voici une chaîne
```

- Guillemets simples

```
$a='chaîne';
$b='voici une $a';
```

```
chaîne
voici une $a
```



25

## Substitution/interprétation de variables dans les chaînes

- Syntaxe *HereDoc*

```
$a='chaîne';
$b=<<<IDENTIFIANT
voici une $a
sur deux lignes;-)
IDENTIFIANT;
```

```
chaîne
```

```
voici une chaîne
sur deux lignes;-)
```

- **La notation <<<IDENTIFIANT est utilisée pour créer une chaîne de caractères délimitée (heredoc) en PHP. Elle permet de définir une chaîne de caractères multiligne tout en préservant la mise en forme du texte, y compris les sauts de ligne et les espaces.**
- **IDENTIFIANT** au choix, après **<<<** et seule sur la ligne à la fin



26

## La commande echo

- echo permet d'envoyer du texte au navigateur du client (« écrire » la page au format HTML résultant de l'interprétation de PHP)
  - `echo 'Bonjour';`
  - `$nom = 'Marcel';`
  - `echo "Bonjour $nom";`
- Exactement, permet d'envoyer des octets au navigateur du client
  - Contenu HTML, XML, CSS, JavaScript, …
  - Données d'une image
  - Contenu d'un ficher PDF, text, etc.
- Précisément, remplit le corps de la réponse HTTP et engendre la production et l'envoi de la réponse HTTP complète



27

## Hello world !

Interprétation du code PHP sur le serveur et transmission du résultat au client dans le corps d'une réponse HTTP

Navigateur

```
<html>
  <head>
    <title>hello</title>
  </head>
<body>
Hello world!
 </body>
</html>
```

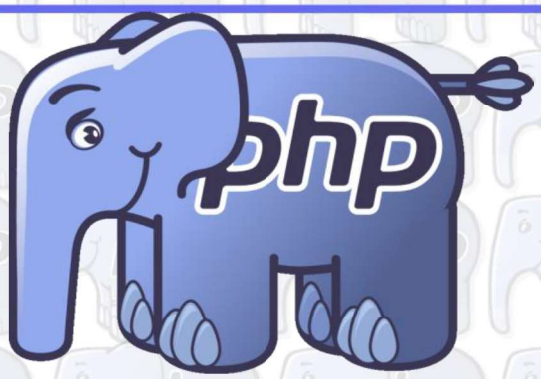
Serveur

```
<?php
$debut = <<<HTML
<html>
  <head>
    <title>hello</title>
  </head>
  <body>\n
HTML;
$corps = "Hello world!\n";
$fin   = <<<HTML
 </body>
</html>
HTML;
/* Envoi au client */
echo  $debut . $corps . $fin;
```

**Impossible de voir le code PHP depuis le navigateur !!**



28



# OPÉRATEURS





## Concaténation de chaînes

- Permet d'assembler plusieurs chaînes
- Réalisé grâce à l'opérateur point : « . »

```
$x = 'Bonjour ' . 'Marcel';
```

→ $x vaut 'Bonjour Marcel'

```
$nb = 6*2;
$y = 'Acheter ' . $nb . ' œufs';
```

→ $y vaut 'Acheter 12 œufs'

## Les opérateurs arithmétiques

| | |
|---|---|
| `$a + $b` | Somme |
| `$a - $b` | Différence |
| `$a * $b` | Multiplication |
| `$a / $b` | Division |
| `$a % $b` | Modulo (Reste de la division entière) |
| `$a ** $b` | Résultat de l'élévation de `$a` à la puissance `$b`. |



## Les opérateurs d'in- et de dé-crémentation pré- et post-fixés

| | |
|---|---|
| `$a++` | Retourne la valeur de `$a` puis augmente la valeur de `$a` de 1 |
| `++$a` | Augmente la valeur de `$a` de 1 puis retourne la nouvelle valeur de `$a` |
| `$a--` | Retourne la valeur de `$a` puis diminue la valeur de `$a` de 1 |
| `--$a` | Diminue la valeur de `$a` de 1 puis retourne la nouvelle valeur de `$a` |

## Les opérateurs de comparaison

| | |
|---|---|
| `$a == $b` | Vrai si égalité entre les valeurs de `$a` et `$b` |
| `$a != $b` | Vrai si différence entre les valeurs de `$a` et `$b` |
| `$a < $b` | Vrai si `$a` inférieur à `$b` |
| `$a > $b` | Vrai si `$a` supérieur à `$b` |
| `$a <= $b` | Vrai si `$a` inférieur ou égal à `$b` |
| `$a >= $b` | Vrai si `$a` supérieur ou égal à `$b` |
| `$a === $b` | Vrai si `$a` et `$b` identiques (valeur et type) |
| `$a !== $b` | Vrai si `$a` et `$b` différents (valeur ou type) |

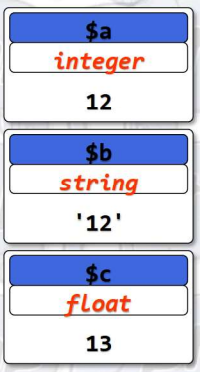


## Comparaison large / stricte

```php
$a = 12; $b = '12'; $c = 13.0;

var_dump($a ==  $b);
var_dump($a ==  $c);
var_dump($c ==  $b);
var_dump($a !=  $b);
var_dump($a !=  $c);
var_dump($c !=  $b);
var_dump($a === $b);
var_dump($a === $c);
var_dump($c === $b);
var_dump($a !== $b);
var_dump($a !== $c);
var_dump($c !== $b);
```

```
boolean true
boolean false
boolean false
boolean false
boolean true
boolean true
boolean false
boolean false
boolean false
boolean true
boolean true
boolean true
```



## Les opérateurs logiques

| | |
|---|---|
| Expr1 **and** Expr2 | Vrai si Expr1 et Expr2 sont vraies |
| Expr1 **&&** Expr2 | Idem |
| Expr1 **or** Expr2 | Vrai si Expr1 ou Expr2 sont vraies |
| Expr1 **\|\|** Expr2 | Idem |
| Expr1 **xor** Expr2 | Vrai si Expr1 ou Expr2 sont vraies mais pas les deux |
| **!** Expr1 | Vrai si Expr1 est non vraie |



35

## Les opérateurs logiques : dangers de **and** et **or** !

```php
<?php

$a = true && false;
var_dump($a);

$b = true and false;
var_dump($b);

$c = false || true;
var_dump($c);

$d = false or true;
var_dump($d);
```



36

## Les opérateurs logiques : dangers de **and** et **or** !

```php
<?php

$a = true && false;
var_dump($a); // false

$b = true and false;
var_dump($b); // true

$c = false || true;
var_dump($c); // true

$d = false or true;
var_dump($d); // false
```



37

## Priorité des opérateurs

| Opérateurs | |
|---|---|
| ** | Exponentiation (puissance). |
| ++ -- | Incrémentation et décrémentation |
| ! | Négation logique |
| * / % | Multiplication, division, modulo |
| + - | Addition, soustraction |
| . | Concaténation de chaînes de caractères |
| < <= > >= | Opérateurs de comparaison |
| =, +=, -=, *=, /=, %= | Opérateurs d'affectation |

En cas de doute,
utilisez les parenthèses ;-)





38

## Exercice

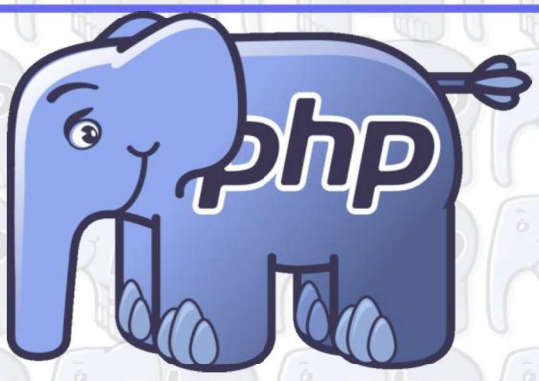
Énoncé :
Écrivez un programme en PHP qui :
1.Choisit un nombre entier.
2.Calcule le carré de ce nombre.
3.Affiche le nombre choisi et son carré.

**Exemple de sortie attendue :**

```
nombre choisi : 24
voici son carré : 576
```



39




## STRUCTURES DE CONTRÔLE



40

## Structure de contrôle Si…Alors…Sinon…

```
if (condition) {
  /* Bloc d'instructions exécuté
     si la condition est vraie */

} else {
  /* Bloc d'instructions exécuté
     si la condition est fausse */

}
```

Optionnel



41

## Exercice

Écrivez un programme en PHP qui :
Définit un montant brut.
Applique un taux de remise en fonction du montant brut :
Si le montant est inférieur à 2000, le taux est de 0%.
Si le montant est compris entre 2000 et 5000 inclus, le taux est de 1%.
Si le montant est supérieur à 5000, le taux est de 2%.
Calcule la remise et soustrait cette remise du montant brut.
Affiche le montant brut et le montant après remise.

**Exemple de sortie attendue :**

```
montant brut : 3000 Dh
Le montant de la remise : 30 Dh
montant après remise : 2970 Dh
```



42

## Structure de contrôle Tant que… faire…

```
while (condition) {
  /* Bloc d'instructions répété
     tant que la condition est vraie */
}
```

---

```
do {
  /* Bloc d'instructions
     exécuté une fois
     puis répété tant que la condition est vraie */
} while (condition);
```



43

## Exercice

Écrivez un programme en PHP qui :
Définit un mot avec un espace final.
Utilise une boucle do...while pour parcourir chaque caractère du mot.
Affiche les voyelles (a, e, i, o, u, y) rencontrées dans le mot.
Arrête la lecture des caractères lorsqu'on atteint la fin du mot.

**Exemple de sortie attendue :**

```
les voyelles rencontrées dans le mot
"anticonstitutionnellement" sont : a,i,o,i,u,i,o,u,e,e,e
```



44

## Structure de contrôle Pour…

```
for (avant; condition; fin_chaque_itération) {
  /* Bloc d'instructions répété
     tant que la condition est vraie */
}
```

**Équivalent à :**

```
avant;
while (condition) {
  /* Bloc d'instructions répété
     tant que la condition est vraie */
  fin_chaque_itération;
}
```



45

## Exercice

Écrivez un programme en PHP qui :
Initialiser un nombre entier positif.
Utilise une boucle « FOR » pour afficher les nombres pairs de 1 jusqu'à ce nombre.
Affiche chaque nombre pair sur une nouvelle ligne.

**Exemple de sortie attendue :**

```
Pour le nombre 10, le programme doit afficher :
2
4
6
8
10
```



46

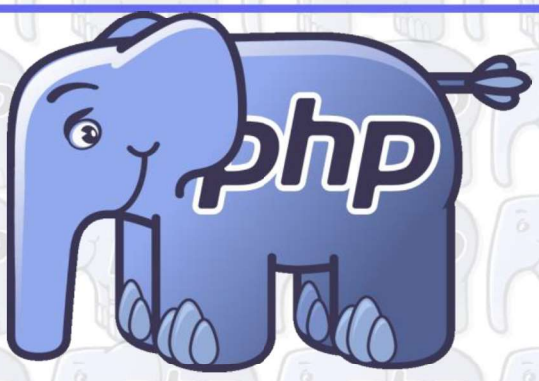
## Structure de contrôle selon…

```
switch (test_expr) {
  case cond_expr_1:
    instructions exécutées si test_expr == cond_expr_1
    break;  /* Sortie du switch */
  case cond_expr_2:
    break;  /* Sortie du switch */
  case cond_expr_3:
    instructions exécutées si test_expr == cond_expr_2
                           ou test_expr == cond_expr_3
    break; /* Sortie du switch */
  default:
    instructions exécutées dans tous les cas
}
```

- **Break** permet de sortir d'une structure de contrôle
- Cas rendus exclusifs si **break** présent pour chaque **case**
- Fonctionne exactement comme une série de **if/else** avec comparaison large



47




# TABLEAUX



48

## Principes généraux

- Un tableau PHP est en réalité une carte ordonnée
- Une carte associe des valeurs à des clés
- Un tableau PHP peut être vu comme une série ordonnées de valeurs (peu importe leur type)
- Chaque valeur est étiquetée, repérée par une clé `int` ou `string`
- Ils sont dits associatifs

| Clé | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| Valeur | 12 | 15.2 | '42' | NULL |

| Clé | 'début' | 12 | 'a' | 'valeur' |
|---|---|---|---|---|
| Valeur | false | 3 | 16 | 'bonjour' |





## Tableaux « classiques » : indexés

- Création / initialisation:

```php
$tab1 = array(12, 'fraise', 2.5);

$tab1 = [ 12, 'fraise', 2.5 ];

$tab2[] = 12;
$tab2[] = 'fraise';
$tab2[] = 2.5;

$tab3[0] = 12;
$tab3[1] = 'fraise';
$tab3[2] = 2.5;
```

| Clé | Valeur |
|---|---|
| 0 | 12 |
| 1 | 'fraise' |
| 2 | 2.5 |

## Tableaux associatifs

```
$tab5[ 'un'   ] = 12;
$tab5['trois'] = 'fraise';
$tab5['deux' ] = 2.5;
$tab5[  42    ] = 'el5';

$tab6 = array(  'un'   => 12,
              'trois'  => 'fraise',
              'deux'   => 2.5,
                42     => 'el5');

$tab7 = [ 'un'    => 12, 'trois' => 'fraise',
          'deux'  => 2.5, 42     => 'el5' ];
```

| Clé | Valeur |
|---|---|
| 'un' | 12 |
| 'trois' | 'fraise' |
| 'deux' | 2.5 |
| 42 | 'el5' |



51

## Structure de contrôle « Pour chaque… »

Des tableaux associatifs ne peuvent PAS être parcourus grâce aux indices des cases contenant les éléments…

```
foreach ($tableau as $element) {
  /* Bloc d'instructions répété
     pour chaque élément de $tableau */
  /* Chaque élément de $tableau
     est accessible grâce à $element */
}
```



52

## Parcours de tableau : **foreach**

```
…
$tab4[0] = 12;
$tab4[6] = 'fraise';
$tab4[2] = 2.5;
$tab4[5] = 'el5';
foreach($tab4 as $v)
{
    echo "<p>Val: $v\n";
}
…
```



```
…
<p>Val: 12\n
<p>Val: fraise\n
<p>Val: 2.5\n
<p>Val: el5\n
…
```



53

## Structure de contrôle « Pour chaque… »

```
foreach($tableau as $cle => $element) {

/* Bloc d'instructions répété
   pour chaque élément de $tableau */

/* Chaque élément de $tableau
   est accessible grâce à $element */

/* La clé d'accès à chaque élément
   est donnée par $cle */

}
```



54

## Parcours de tableau

```
<!doctype html>
<html lang="fr">
  <head><title>foreach clé</title></head>
<body>
<?php
$tab6 = ['un'=>12,'trois'=>
'fraise','deux'=>2.5,42=>'el5'];
foreach ($tab6 as $cle => $val)
echo $cle ," : ", $val,"<br>";
?>
</body>
</html>
```



```
…
un : 12
trois : fraise
deux : 2.5
42 : el5
…
```



55

## Exercice

Créez un script PHP qui fait les choses suivantes :

1. Crée un tableau contenant les noms de cinq fruits.
2. Affiche le nombre total de fruits dans le tableau.
3. Affiche chaque fruit sur une nouvelle ligne.
4. Ajoute un nouveau fruit (kiwi) au tableau.
5. Affiche le tableau mis à jour (avec simple for et avec foreach).
6. Vérifier si Banane existe dans le tableau, et afficher sa position s'elle existe ?
7. Remplacer Mangue par Poire

```
$fruits = ["Pomme", "Banane", "Orange", "Mangue", "Ananas"];
```



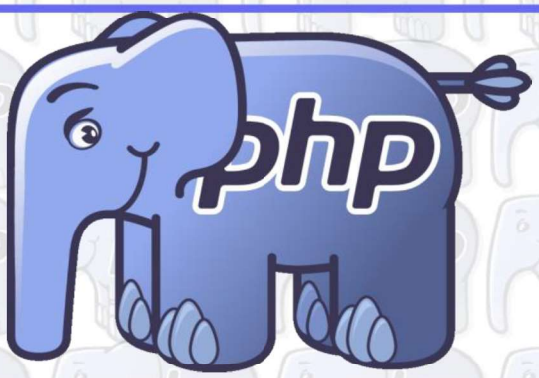
56

## Exercice

1. Créer un tableau associatif contenant cinq fruits avec leur couleur respective. Et afficher le nombre total de fruits dans le tableau.
2. Afficher chaque fruit et sa couleur sur une nouvelle ligne.
3. Ajouter un nouveau fruit avec sa couleur au tableau (Kiwi de couleur vert). Et Afficher le tableau mis à jour.
4. Vérifier si le fruit “Pomme” existe dans le tableau ?
   Utilisez array_key_exists("Pomme", $fruits)
5. Supprimer le fruit “Ananas” du tableau
   Utilisez unset($fruits["Ananas"])
6. Écrire une fonction qui prend une couleur et le tableau des fruits en paramètre et retourne tous les fruits de cette couleur.
7. Générer un tableau HTML pour afficher les fruits et leurs couleurs.
8. Écrire une fonction pour inverser les clés et les valeurs du tableau (les couleurs deviennent les clés et les fruits les valeurs) et afficher le nouveau tableau en HTML.

```
$fruits = [
    "Pomme" => "Rouge",
    "Banane" => "Jaune",
    "Orange" => "Orange",
    "Mangue" => "Jaune",
    "Ananas" => "Marron" ];
```






## FONCTIONS

## Fonctions utilisateur

- Description d'une fonctionnalité dépendant éventuellement de paramètres et retournant éventuellement un résultat
- Définition

```
function moyenne($a, $b)
{
    return ($a + $b) / 2;
}
```

- Utilisation

```
$resultat = moyenne(2, 4);
echo $resultat; // vaut 3
```



59

## Fonctions utilisateur PHP ≤ 4

- Valeur de retour

```
function moyenne($a, $b)
{  …  }
```

- Arguments

```
function moyenne( $a, $b)
{  …  }
```

Typage faible de PHP : Aucune information

Typage faible de PHP : Aucune information

- Variables définies dans une fonction

➔ Visibles et accessibles dans la fonction



60

## Fonctions utilisateur PHP ≥ 5

- Valeur de retour

```
function moyenne($a, $b): int
{   …   }
```

Typage possible

Le typage sera obligatoire, c'est un garde-fou pour les développeurs

- Arguments

```
function moyenne(int $a, int $b): int
{   …   }
```

- Fonctionnement identique pour les méthodes
- Possible de typer paramètre et valeur de retour



61

## Mode de passage des arguments (types natifs)

```php
<?php
function permutation(int $x, int $y): void {
   echo "permutation...";
   $t = $x;
   $x = $y;
   $y = $t;
}
$a = 12;
$b = 210;
echo "\$a = $a";
echo "\$b = $b";
permutation($a, $b);
echo "\$a = $a";
echo "\$b = $b";
```

Permutation impossible : Passage des arguments des fonctions par valeur

```
$a = 12
$b = 210
permutation...
$a = 12
$b = 210
```



62

## Mode de passage des arguments (types natifs)

```php
<?php
function permutation(int &$x, int &$y): void {
   echo "permutation...";
   $t = $x;
   $x = $y;
   $y = $t;
}
$a = 12;
$b = 210;
echo "\$a = $a";
echo "\$b = $b";
permutation($a, $b);
echo "\$a = $a";
echo "\$b = $b";
```

Permutation réussie

```
$a = 12
$b = 210
permutation...
$a = 210
$b = 12
```

```php
function calculerSomme(int $a, int $b): int {
        return $a + $b;
}
```



63

## Arguments par défaut des fonctions

- Valeur par défaut d'un argument s'il n'a pas été défini lors de l'appel de la fonction

```php
function bonjour(string $nom='Invité'): void
{
  echo "Bonjour cher $nom";
}
```

- Utilisation

```php
bonjour();
```

```
Bonjour cher Invité
```

```php
bonjour('Karim');
```

```
Bonjour cher Karim
```



64

## Gestion des erreurs

- Dans certains cas, il n'est ni possible ni utile de poursuivre l'exécution du code PHP (variables non définies, valeurs erronées, échec de connexion, …)
- Arrêt brutal de l'exécution du code:
  - `die(message)`
  - `exit(message)`

  Envoie *message* au navigateur et termine l'exécution du script courant



65

## Opérateur de contrôle d'erreur en phase de développement

```
$v = file("dummy.txt")
   or die("Problème de lecture");
```

Fichier absent

```
Warning: file(dummy.txt): failed to open stream: No such
file or directory in dummy.php on line 8
Problème de lecture
```

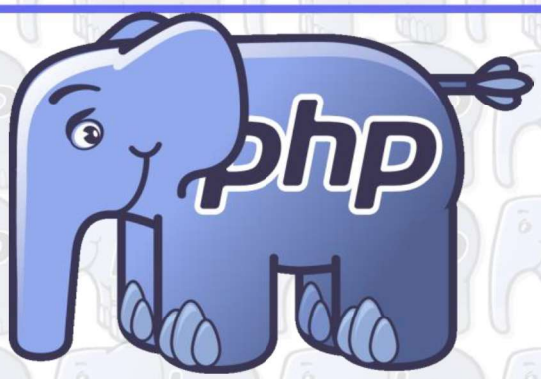
```
$v = @file("dummy.txt")
   or die("Problème de lecture");
```

Fichier absent

```
Problème de lecture
```



66



# DONNÉES DE FORMULAIRES



67

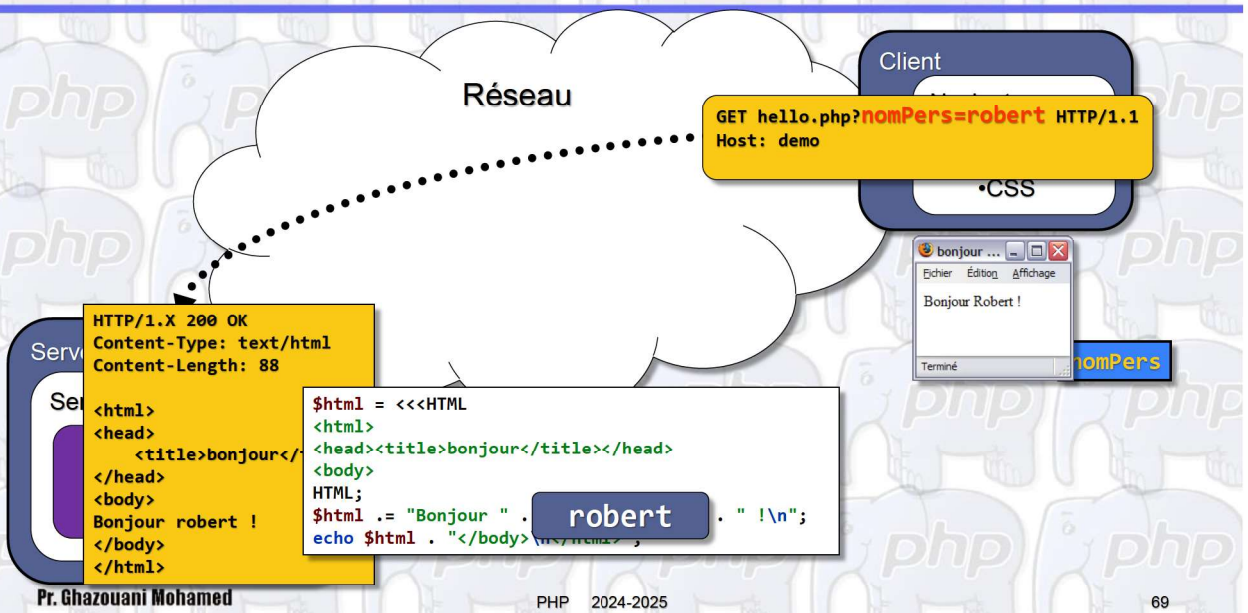
## Traitement des données de formulaires

- PHP permet de traiter les données saisies grâce à un formulaire HTML si le champ **ACTION** du formulaire désigne une ressource PHP du serveur.
- Après récupération par le serveur Web, les données sont contenues dans l'une des variables superglobales de type tableau associatif `$_GET` ou `$_POST` selon la méthode de la requête (ou `$_REQUEST` qui reçoit le contenu de `$_GET` et `$_POST`).
- La valeur peut être trouvée grâce à une clé qui porte le même nom que le champ du formulaire de la page HTML de saisie.



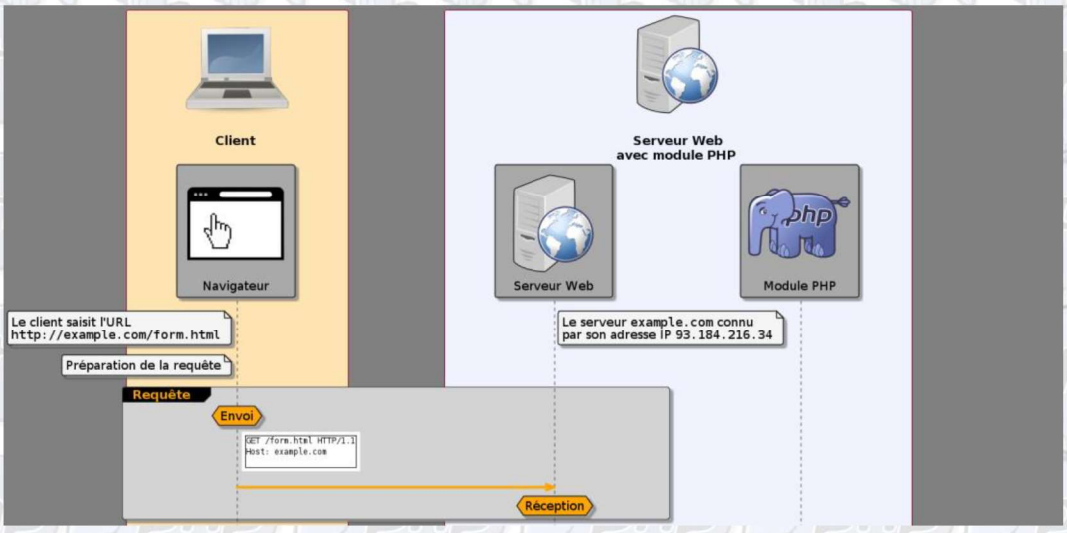
68

# Traitement des données de formulaires



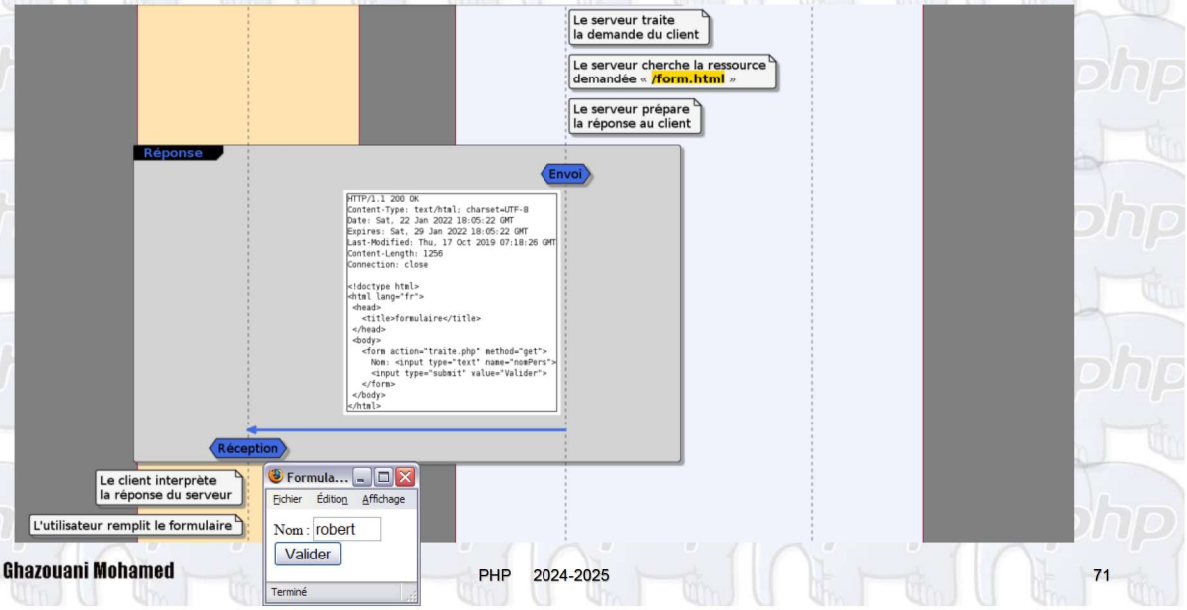

69

# Formulaire et traitement des données (1)




70

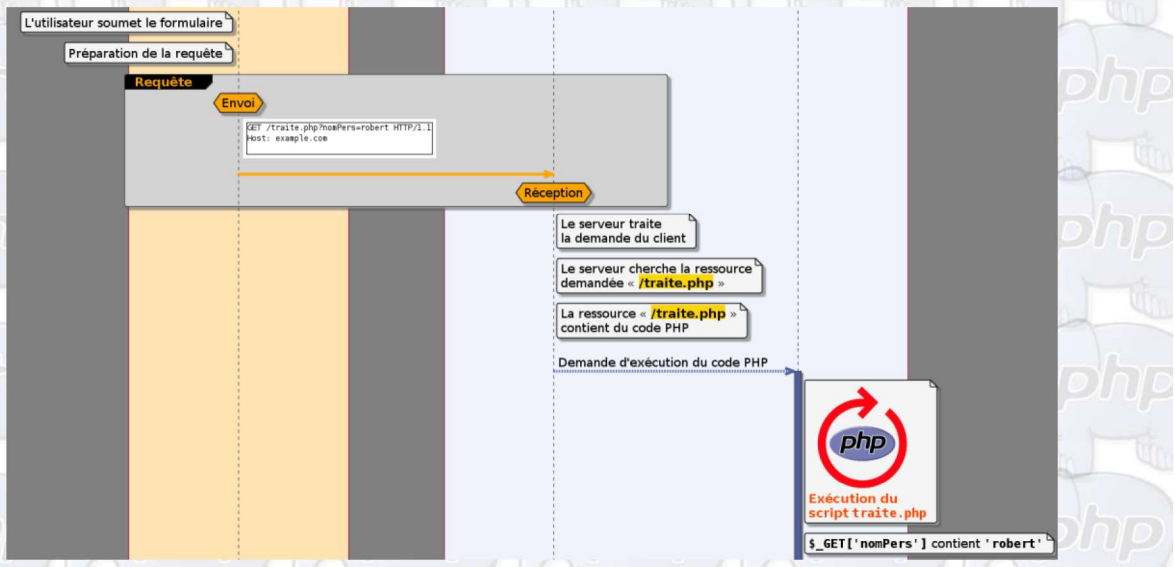
# Formulaire et traitement des données (2)






71

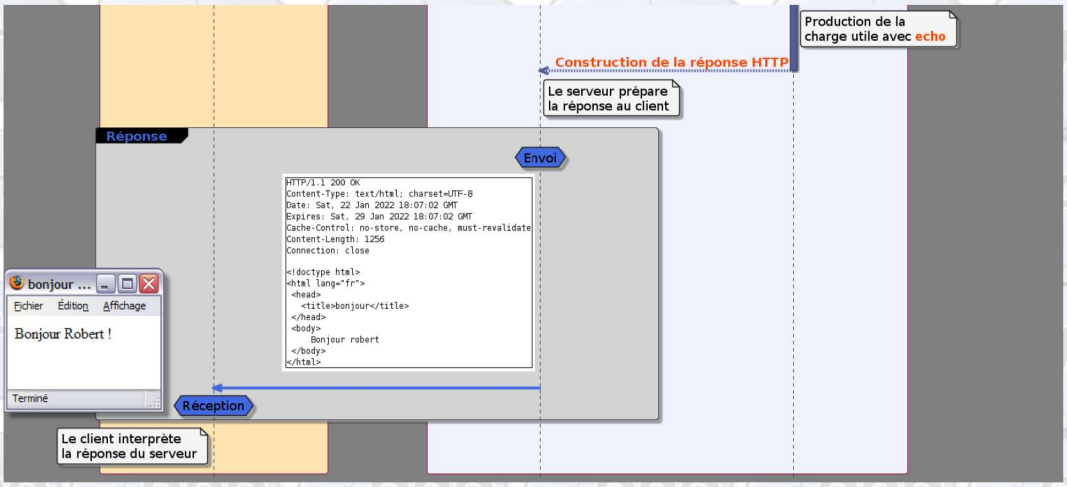
# Formulaire et traitement des données (3)






72

# Formulaire et traitement des données (4)






73

# Exemple – Formulaire HTML

```
<!doctype html>
<html lang="fr">
<head>
    <title>formulaire</title>
</head>
<body>
<form action="traite.php" method="get">
    Nom: <input type="text" name="nomPers">
    <input type="submit" value="Valider">
</form>
</body>
</html>
```



74

## Exemple – Traitement en PHP

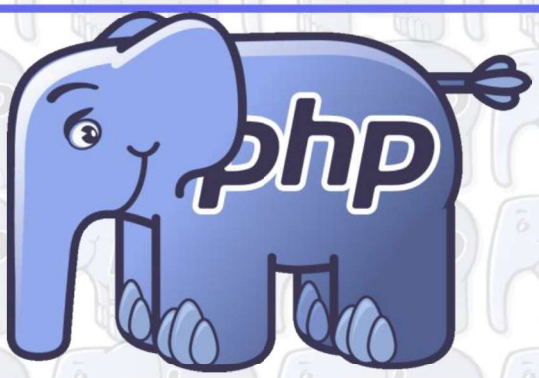
```
<html lang="fr">
<head><title>Bonjour</title></head>
<body>
<?php
if ($_SERVER["REQUEST_METHOD"]=="GET")
if (isset($_GET["nomPers"]))
    if (!empty($_GET["nomPers"]))
        echo "bonjour ".$_GET["nomPers"];
    else
        echo "Vous n'avez rien saisi";
else
    echo "Ce champs n'est pas défini";
?>
</body>
</html>
```

$_GET['nomPers'] est-il défini ?

$_GET['nomPers'] est-il vide ?



75

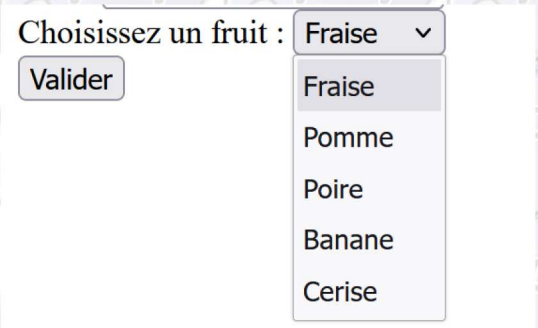



## DONNÉES DE FORMULAIRES : QUELQUES EXEMPLES



76

## Formulaires contenant des champs « SELECT »






77

## Formulaires contenant des champs « SELECT unique»

```
<!doctype html>
<html lang="fr">
<head>
   <title>Formulaire de saisie des fruits</title>
</head>
<body>
   <form action="valide3.php" method="get">
        Choisissez des fruits :
        <select name="fruit">
                <option>Fraise
                <option>Pomme
                <option>Poire
                <option>Banane
                <option>Cerise
        </select>
        <input type="submit" value="envoyer">
   </form>
</body>
</html>
```

Envoyer

valide3.php?fruit=Pomme



78

## Formulaires contenant des champs « SELECT multiple»

```
<!doctype html>
<html lang="fr">
<head>
   <title>Formulaire de saisie des fruits</title>
</head>
<body>
   <form action="valide3.php" method="get">
        Choisissez des fruits :
        <select name="fruits" multiple>
                <option>Fraise</option>
                <option>Pomme</option>
                <option>Poire</option>
                <option>Banane</option>
                <option>Cerise</option>
        </select>
        <input type="submit" value="envoyer">
   </form>
</body>
</html>
```

Envoyer

valide3.php?fruits=Pomme&fruits=Poire

???



79

## Formulaires contenant des champs « SELECT multiple»

```
<!doctype html>
<html lang="fr">
<head>
   <title>Formulaire de saisie des fruits</title>
</head>
<body>
   <form action="valide3.php" method="get">
        Choisissez des fruits :
        <select name="fruits[]" multiple>
                <option>Fraise</option>
                <option>Pomme</option>
                <option>Poire</option>
                <option>Banane</option>
                <option>Cerise</option>
        </select>
        <input type="submit" value="envoyer">
   </form>
</body>
</html>
```

Envoyer

valide3.php?fruits%5B%5D=Pomme&fruits%5B%5D=Poire

valide3.php?fruits[]=Pomme&fruits[]=Poire



80

## Traitement des données des champs « SELECT »

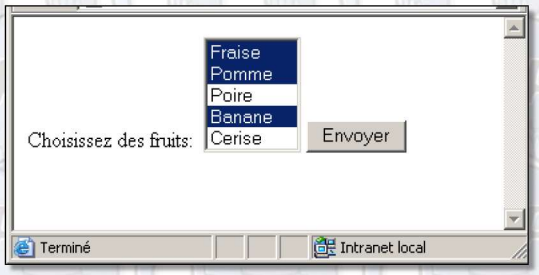
```
<!doctype html>
<html lang="fr">
<head>
   <title>Liste de fruits</title>
</head>
<body>
<?php
   if (isset($_GET['fruits']))
   {
        foreach($_GET['fruits'] as $fruit)
            echo "<p>Vous avez choisi $fruit";
   }
   else
        echo "<p>Vous n'avez pas choisi de fruit";
?>
</body>
</html>"
```

$_GET['fruits'] est un tableau



81

## Résultat









82

## Formulaires contenant des champs « CHECKBOX »






83

## Formulaires contenant des champs « CHECKBOX »

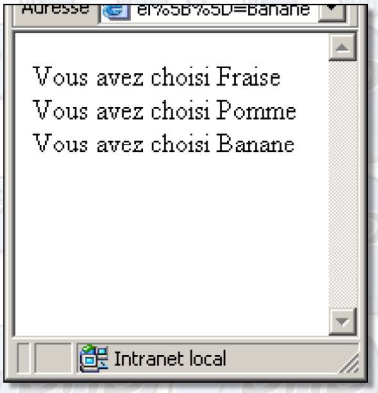
```
<!doctype html>
<html lang="fr">
 <head>
   <title>Formulaire de saisie des fruits</title>
 </head>
 <body>
  <form name="formu" action="valide3.php" method="get">
   Choisissez des légumes :
   <label><input type="checkbox" name="legumes[]" value="Carotte">Carotte</label>
   <label><input type="checkbox" name="legumes[]" value="Tomate">Tomate</label>
   <label><input type="checkbox" name="legumes[]" value="Laitue">Laitue</label>
   <label><input type="checkbox" name="legumes[]" value="Brocoli">Brocoli</label>
   <label><input type="checkbox" name="legumes[]" value="Épinard">Épinard</label>
  <input type="submit" value="Envoyer">
   </form>
 </body>
</html>
```



84

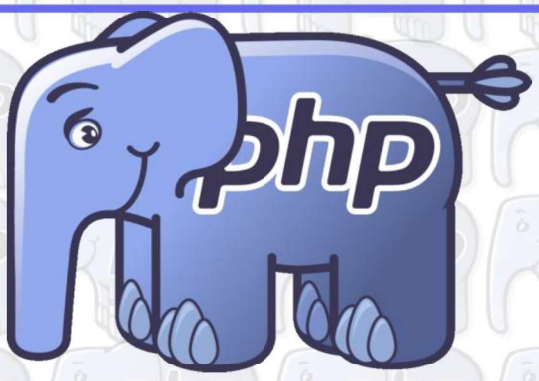
## Résultat






85



## DONNÉES DE FORMULAIRES : VUE UTILISATEUR / VALEUR TRANSMISE



86

## Formulaires contenant des champs « SELECT unique»

```
<!doctype html>
<html lang="fr">
<head>
   <title>Formulaire de saisie des fruits</title>
</head>
<body>
   <form action="valide3.php" method="get">
        Choisissez des fruits :
        <select name="sel">
               <option value="1">Fraise
               <option value="2">Pomme
               <option value="3">Poire
               <option value="4">Banane
               <option value="5">Cerise
        </select>
        <input type="submit" value="envoyer">
   </form>
</body>
</html>
```



87

## Formulaires contenant des champs « CHECKBOX »

```
<!doctype html>
<html lang="fr">
 <head>
   <title>Formulaire de saisie des fruits</title>
 </head>
 <body>
  <form name="formulaire" action="valide3.php" method="get">
   Choisissez des fruits :
   <label><input type="checkbox" name="fruits[]" value="1">Fraise</label>
   <label><input type="checkbox" name="fruits[]" value="2">Pomme</label>
   <label><input type="checkbox" name="fruits[]" value="3">Poire</label>
   <label><input type="checkbox" name="fruits[]" value="4">Banane</label>
   <label><input type="checkbox" name="fruits[]" value="5">Cerise</label>
   <input type="submit" value="Envoyer">
  </form>
 </body>
</html>
```



88

## Sécurisation des Données Utilisateur en PHP
## Utilisation de htmlspecialchars() et strip_tags()

### htmlspecialchars()

**Objectif** : Convertir les caractères spéciaux en code HTML pour qu'ils ne soient pas interprétés comme du code HTML.

**Quand l'utiliser** : Quand vous souhaitez afficher le texte tel qu'il est, mais empêcher l'exécution de code HTML pour éviter des risques de sécurité (XSS - Cross-Site Scripting).

**Exemple :**

```
$commentaire = '<script>alert("Hacked!");</script> <strong> Bonjour !</strong>';
echo htmlspecialchars($commentaire);
// Sortie : <script>alert("Hacked!");</script> <strong> Bonjour !</strong>
```



89

## Sécurisation des Données Utilisateur en PHP
## Utilisation de htmlspecialchars() et strip_tags()

### strip_tags()

- **Objectif** : Supprimer toutes les balises HTML et PHP d'une chaîne de caractères.
- **Quand l'utiliser** : par exemple dans un champ de saisie où seul le texte est accepté.

**Exemple** :

```
$commentaire = '<script>alert("Hacked!");</script> <strong> Bonjour !</strong>';
echo strip_tags($commentaire); // Sortie : alert("Hacked!"); Bonjour !
```



90

## Sécurisation des Données Utilisateur en PHP

Voici quelques fonctions PHP couramment utilisées pour manipuler les chaînes de caractères et vérifier les types de données :

- Supprimer les espaces de gauche et de droite : $chaine_trim = trim($chaine)
  ltrim($chaine) et rtrim($chaine)
- Convertir en majuscules : $chaine_maj = strtoupper($chaine)
- Convertir en minuscules : $chaine_min = strtolower($chaine)
- Convertir la première lettre en majuscule : ucfirst($chaine)
- Longueur d'une chaîne : strlen($chaine)
- Diviser une chaîne en tableau : $tableau = explode(" ", $chaine)

```
$chaine = "Bonjour tout le monde";
$tableau = explode(" ", $chaine);
print_r($tableau); // Sortie : Array ( [0] => Bonjour [1] => tout
[2] => le [3] => monde )
```

- Joindre un tableau en chaîne : $chaine = implode(' ', $tableau)

```
$tableau = ['Bonjour', 'tout', 'le', 'monde'];
$chaine = implode(' ', $tableau);
echo $chaine; // Sortie : "Bonjour tout le monde"
```



91

## Sécurisation des Données Utilisateur en PHP

- Vérifier que c'est un nombre : (is_numeric($valeur))

```
$valeur1 = "12345"; $valeur2 = 12345; $valeur3 = "123.45";
$valeur4 = 123.45; $valeur5 = "abc";
```

- Vérifier si une variable est un entier : (is_int($valeur))

```
$valeur1 = "12345"; $valeur2 = 12345; $valeur3 = "123.45";
$valeur4 = 123.45; $valeur5 = "abc";
```

- Vérifier si une variable est une chaîne : (is_string($valeur))
- Vérifier si une variable est un : (is_array($valeur)), (is_bool($valeur)), (is_float($valeur))
- filter_var($_POST['nombre'], FILTER_VALIDATE_FLOAT)
  FILTER_VALIDATE_EMAIL/INT/URL
- filter_var($_POST['nombre'], FILTER_VALIDATE_INT, ["options" => ["min_range" => 0, "max_range" => 10]])



92

## Exercice
## Création d'un Formulaire d'inscription à un événement

**Créer un système d'inscription en ligne pour un événement composé de deux parties : un formulaire de saisie HTML et un script PHP de traitement.**

**Création du formulaire HTML :**

- Créez un fichier evenement.html
- Ajoutez un formulaire avec les champs suivants :
  <form action="evenement.php" method="post">
  - Nom : <input type="text" name="name" >
  - Une liste déroulante pour choisir la session (matin ou l'après-midi)
    - <select name="session"> <option>matin </select>
  - Adresse e-mail : <input type="email" name="email" >
  - Téléphone : <input type="tel" name="telephone" pattern="[0-9]{10}">
  - Commentaire : <textarea name="message" required></textarea>
  - Bouton de soumission <input type="submit" value="Envoyer">

`strip_tags($variable)`

`htmlspecialchars($variable)`

**Traitement des données en PHP :**

- Créez un fichier evenement.php ghazouani.fsbm.td@gmail.com
- Récupérez les données du formulaire en utilisant $_POST.
- Validez les données (vérifiez que tous les champs sont remplis et que l'adresse e-mail est valide en utilisant :

```
if (filter_var($email, FILTER_VALIDATE_EMAIL))
if (preg_match("/^[0-9]{10}$/", $telephone))
```

93

93

## Exercice
## Création d'un Formulaire de Contact en PHP

**Créer un formulaire de contact simple qui permet aux utilisateurs de saisir leur nom, adresse e-mail et message. Les données saisies doivent être traitées et affichées sur une nouvelle page.**

**Création du formulaire HTML :**

- Créez un fichier contact.html
- Ajoutez un formulaire avec les champs suivants :
  <form action="contact.php" method="post">
  - Nom : <input type="text" name="nom" >
  - Adresse e-mail : <input type="email" name="email" >
  - Téléphone : <input type="tel" name="telephone" pattern="[0-9]{10}">
  - Message : <textarea name="message" required></textarea>
  - Bouton de soumission <input type="submit" value="Envoyer">

`strip_tags($commentaire)`

`htmlspecialchars($commentaire)`

**Traitement des données en PHP :**

- Créez un fichier contact.php. ghazouani.fsbm.td@gmail.com
- Récupérez les données du formulaire en utilisant $_POST.
- Validez les données (vérifiez que tous les champs sont remplis et que l'adresse e-mail est valide en utilisant :

```
if (filter_var($email, FILTER_VALIDATE_EMAIL))
if (preg_match("/^[0-9]{10}$/", $telephone))
```

- Affichez les données saisies sur la page contact.php

94

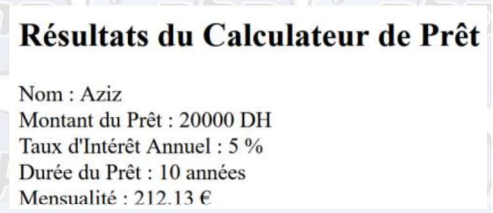
94

## Exercice
## Calculateur de Prêt

**Objectif : Créer une application web pour calculer les mensualités d'un prêt dans un tableau.**

- **Créez un projet intitulé calculPret**
- **Créez un fichier HTML nommé index.html**
- **Créer un formulaire permettant de saisir les informations nécessaires au calcul des mensualités d'un prêt : Nom, Montant du Prêt (Dh), Taux d'Intérêt Annuel (%) et la Durée du Prêt (années)**
- **Créez un fichier PHP nommé traitement.php**
- **Traitez les données soumises et effectuer les calculs nécessaires**
  - **taux mensuel = taux / 100 / 12**
  - **nombre mensualités = duré * 12**
  - **mensualité = montant * taux mensuel / (1 - (1 + taux mensuel) ** -nombre mensualités)**








## Exercice
## Calculatrice Scientifique en PHP

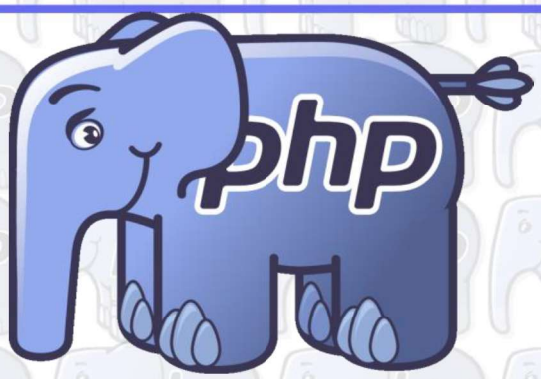
**Création du formulaire HTML :**

- **Créez un fichier calculatrice.html**
- **Ajoutez un formulaire avec les champs suivants :**
  **<form action="calculatrice.php" method="post">**
  - **Premier nombre : <input type="number " name="nombre" >**
  - **Une liste déroulante pour choisir l'opération avec les options suivantes : Addition (+), Soustraction (-), Multiplication (*), Division (/) (a / b!=0)**
  - **, Puissance (x**y) Racine n-ième (a>=0, b!=0), Modulo (a % b!=0), Factorielle (n!) (n>=0)**
    **<select name="operateur">**
    **<option value="addition">Addition (+)**
  - **Un champ numérique pour le second nombre**
  - **Bouton de soumission <input type="submit" value="Envoyer">**

**Traitement des données en PHP :**

**Afficher les résultats dans un fichier calculatrice.php :**

- **L'opération choisie, les nombres utilisés, le résultat**
- **Les messages d'erreur si nécessaire**



# LECTURE DE BASE DE DONNÉES EN PHP



97

## Lecture de base de données MySql en PHP

```
<?php
    $serveur = "127.0.0.1:3336";
    $utilisateur = "root";
    $motdepass = "";
    $basededonnees = "etudiants";
    $connexion = new mysqli($serveur, $utilisateur, $motdepass, $basededonnees);

    if ($_SERVER["REQUEST_METHOD"] == "POST")
        $nom = $_POST["nom"];
    $age = $_POST["age"];
    $tel = $_POST["telephone"];
    $requete = "INSERT INTO etudiant (nom, age, telephone) VALUES ('$nom', $age,
'$tel')";

    if ($connexion->query($requete)===True)
        echo " votre enregistrement est bien sauvegarder";
    else
        echo "probleme",$connexion->error;
    $connexion->close();
    ?>
```



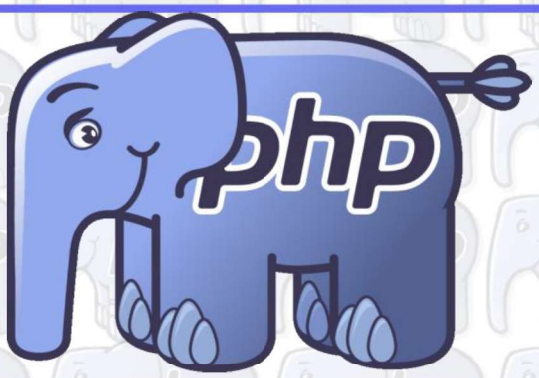
98

# Lecture de base de données MySql en PHP

```php
<?php
$serveur = "localhost:3306"; // Adresse du serveur MySQL ou // 127.0.0.1:3336
$utilisateur = "root"; // Nom d'utilisateur de la base de données
$motDePasse = ""; // Mot de passe de la base de données
$nomBaseDeDonnees = "etudiants"; // Nom de la base de données
$connexion = new mysqli($serveur, $utilisateur, $motDePasse, $nomBaseDeDonnees);
$requete = "SELECT * FROM etudiant";
$resultat = $connexion->query($requete);
if ($resultat) {
    echo "<table>";
    echo "<tr><th>ID</th><th>Nom</th><th>Age</th><th>Téléphone</th></tr>";
    while ($ligne = $resultat->fetch_assoc()) {
        echo "<tr>";
        echo "<td>" . $ligne["id"] . "</td>";
        echo "<td>" . $ligne["nom"] . "</td>";
        echo "<td>" . $ligne["age"] . "</td>";
        echo "<td>" . $ligne["tel"] . "</td>";
        echo "</tr>";
    }
    echo "</table>";
    $resultat->close();
} else
{
    echo "Erreur lors de l'exécution de la requête : " . $connexion->error;
}
// Fermeture de la connexion à la base de données
$connexion->close();
?>
```



99




# MERCI DE VOTRE ATTENTION



100